

taoFPGA

Architecture, Implementation, and Evaluation of a Quantized Transformer Processing Core on an Edge FPGA

Eliran Turgeman 316372002

Shay Rask 314951658

Academic supervisor: **Dr. Leonid Yavits**

Project mentor: **David Freud**

PROJECT 309

1 Background & motivation - the edge-AI compute dilemma

2 Problem statement, objectives & scope

3 Methodology - a verify-every-layer co-design flow

4 Core hardware architecture (systolic array + fixed-point Softmax/GELU)

5 Technical contribution - a synthesis attribute that silently did nothing

6 Results: kernel speedup, silicon verification, honest workload analysis

7 Planning vs. execution, challenges, budget & roadmap

8 Conclusions & recommendations

Background: the edge-AI compute dilemma

- ▶ Transformers are the state of the art in both NLP and computer vision (Vision Transformers, ViT).
- ▶ Their computational profile is hostile to edge hardware:
 - $O(N^2)$ self-attention in the sequence length
 - large dense matrix multiplications in every encoder layer
 - non-linearities (Softmax, GELU) between those matmuls
- ▶ Edge SoCs are simultaneously short on DRAM bandwidth, on-chip memory and power budget - exactly the resources Transformers stress.
- ▶ A general-purpose CPU/GPU wastes energy on control and data movement; a dedicated datapath does not - a computer-engineering problem: co-design the arithmetic, the data movement and the silicon together.

Why an FPGA

- ▶ Custom INT8 datapath, no ISA overhead
- ▶ Hard DSP + BRAM blocks map matmul directly
- ▶ Reprogrammable - an academic-scale ASIC proxy
- ▶ Commodity board: PYNQ-Z2, ~\$199

Builds on prior FPGA transformer accelerators (ME-ViT, FTRANS, systolic-array designs) - same design family, embedded-scale device.

Three ideas the rest of the talk builds on

Self-attention

- ▶ Every token attends to every other token.
- ▶ $Q \cdot K^T \rightarrow \text{scale} \rightarrow \text{Softmax} \rightarrow$ weighted sum of V .
- ▶ Then a position-wise MLP with a GELU.
- ▶ Cost is $O(N^2)$ MACs - the quadratic term.

INT8 quantization

- ▶ Store each value as a signed 8-bit int + a shared power-of-two scale.
- ▶ $x \approx \text{round}(x / \text{scale})$.
- ▶ $\text{INT8} \times \text{INT8} \rightarrow$ wide product $\rightarrow$ round-half-up & saturate to $[-128, 127]$.
- ▶ $\frac{1}{3}$ the memory bandwidth of FP32.

Systolic array

- ▶ 2D grid of tiny multiply-accumulate PEs.
- ▶ Operands stream through neighbours only - no global memory, short wires.
- ▶ Weight-stationary: each PE holds one weight while features stream past.
- ▶ Matches the Transformer's weight-reuse pattern.

These three primitives are the whole talk: matmul + Softmax + GELU, in INT8, on a systolic array.

Problem statement

Run the compute-dominant primitives of a Transformer block - dense matmul plus its Softmax / GELU non-linearities - as a dedicated, numerically-verified INT8 datapath on a commodity 220-DSP edge FPGA, and prove the speed-up is real.

- ▶ FP32 inference is infeasible on an embedded FPGA - every MAC costs more logic, every value costs 4× the bandwidth.
- ▶ A correct accelerator is not enough: it has to be verified against a ground-truth model at every layer (RTL, integration, silicon).
- ▶ And the reported gain must correspond to an operation the hardware can actually perform correctly - not an impressive-looking number that hides a broken computation.

General Design, implement and quantitatively evaluate a complete hardware accelerator for a Transformer block's dominant primitives on a resource-constrained edge FPGA, and demonstrate measurable latency / throughput / energy gains over a CPU baseline.

O1

INT8 weight-stationary 16×16 systolic matmul core with fused fixed-point Softmax and GELU engines, formalised as exact fixed-point equations.

O2

Tolerance-bounded verification: SystemVerilog testbenches + a bit-faithful Python/NumPy golden model carried to silicon.

O3

Integrate the core as an AXI peripheral on a Zynq-7020 (PYNQ-Z2) SoC and bring it up on real hardware.

O4

Close physical timing at 100 MHz and optimise post-route resource / power at signoff.

O5

Benchmark the hardware kernel against the on-chip ARM Cortex-A9 and characterise the result honestly against real ViT workloads.

Scope & boundaries

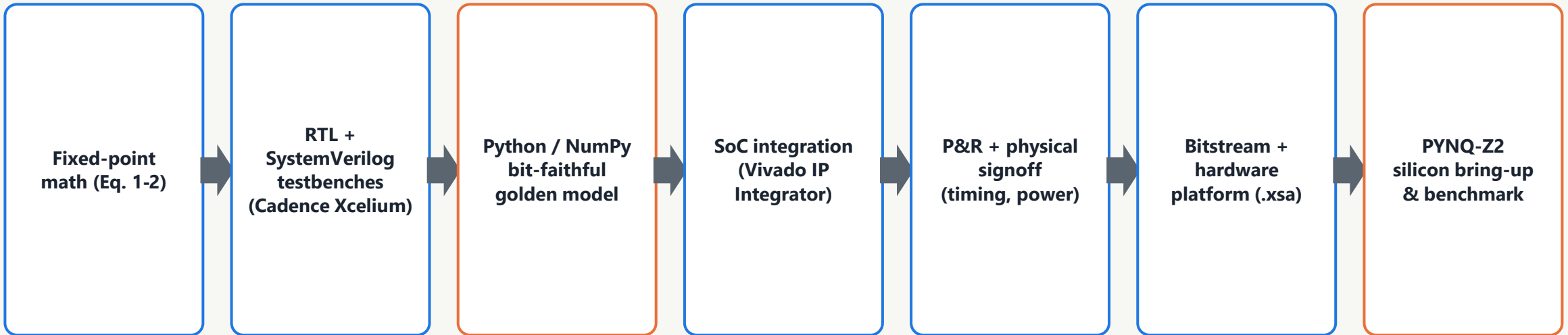
IN SCOPE

- ▶ Fused matmul → Softmax → GELU streaming pipeline
- ▶ INT8, weight-stationary, 16×16 systolic array (256 MACs)
- ▶ Xilinx Zynq-7020 / PYNQ-Z2, 100 MHz
- ▶ RTL flow (SystemVerilog / Verilog), not HLS
- ▶ Kernel-level benchmark vs. the board's ARM Cortex-A9
- ▶ Full physical signoff + bitstream + silicon bring-up

OUT OF SCOPE

- ▶ End-to-end accelerated ViT inference (fixed pipeline maps to no single ViT sub-block - slide 18)
- ▶ Batch processing & the full 768-wide ViT-Tiny MLP (64 KB DMA ceiling)
- ▶ Training / back-propagation
- ▶ Datacenter-class FPGAs and multi-device scaling
- ▶ Physically-instrumented power measurement (signoff-grade estimate used)

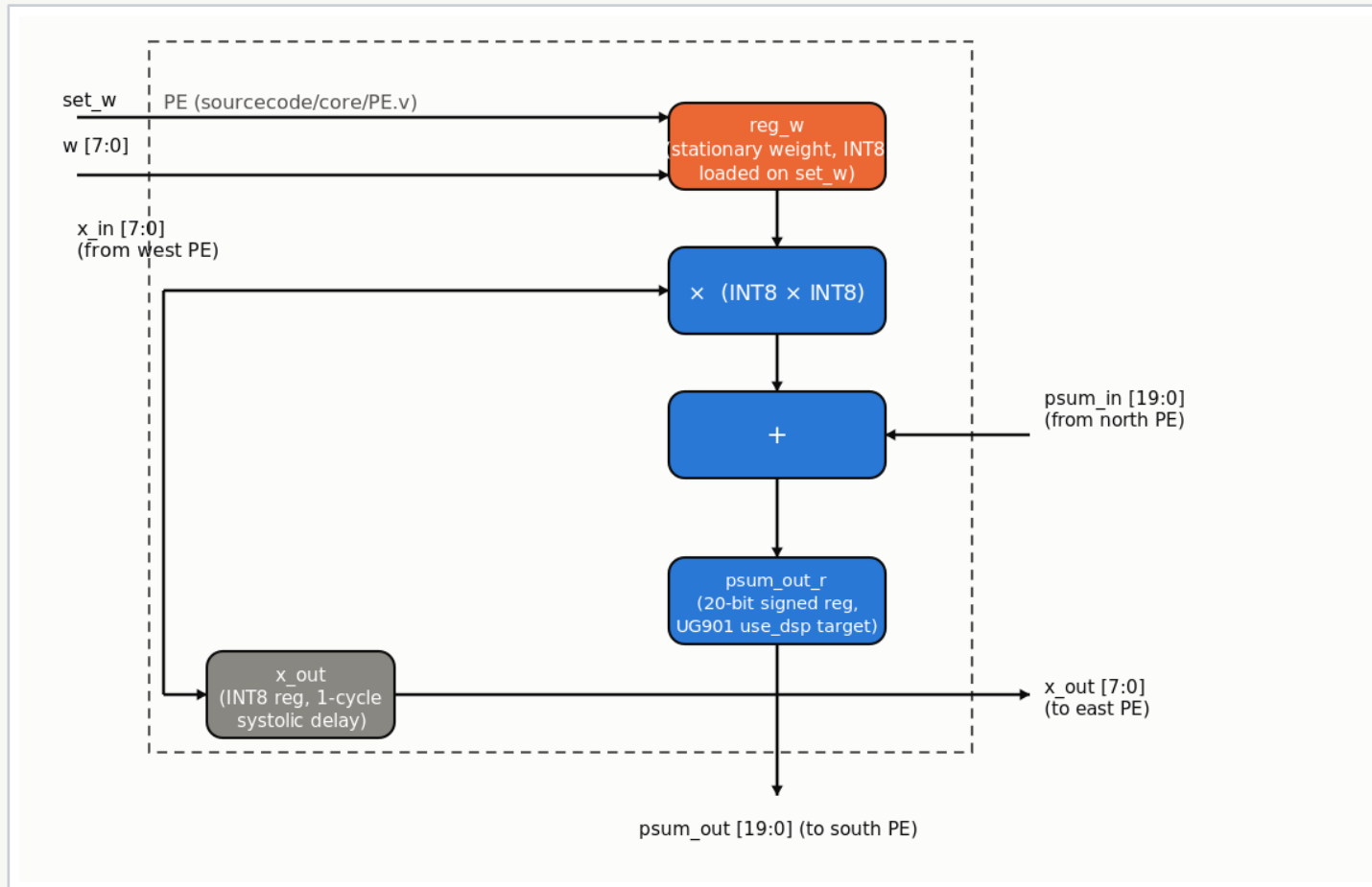
Methodology: a verify-every-layer flow



Governing discipline: every layer is checked against its own reference, never assumed correct because the layer below it passed. The SystemVerilog golden model is re-derived in NumPy and cross-checked against the RTL shifter's exact rounding before it is trusted to validate silicon.

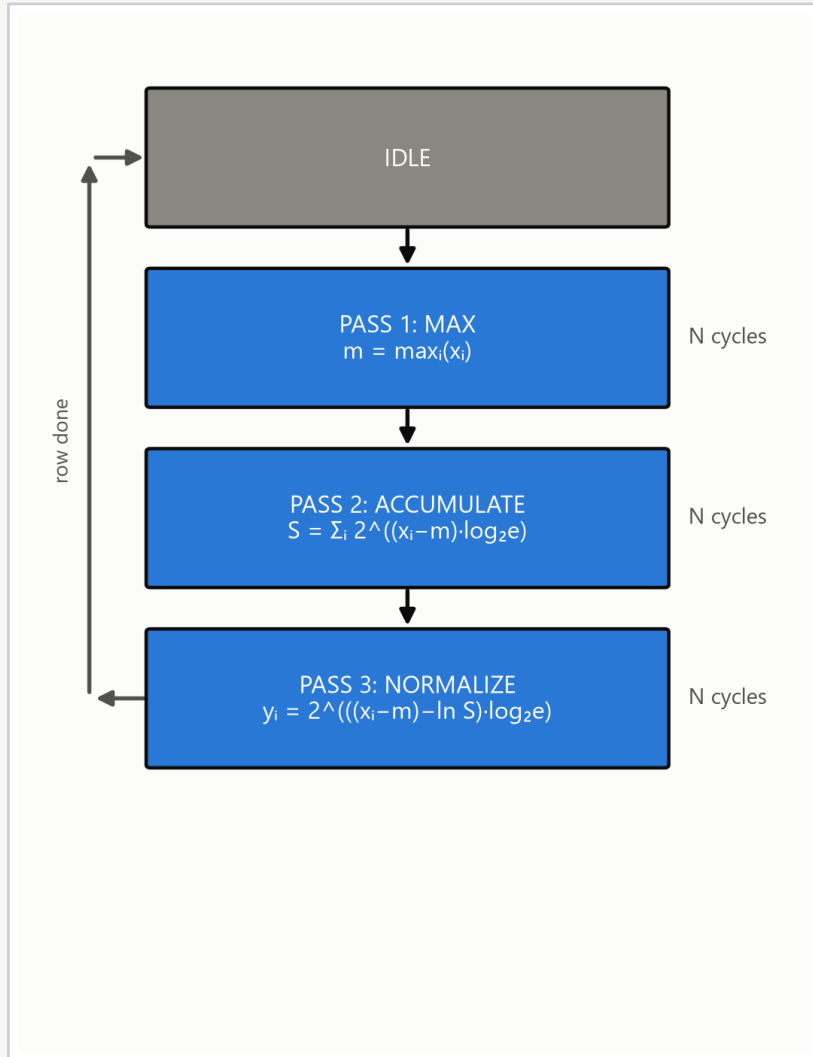
Verification is tolerance-bounded (scale-dependent LSB threshold), not exact-match - to absorb quantization-error compounding across two INT8 boundaries.

Core architecture - the processing element



- ▶ 16×16 grid $\rightarrow$ 256 INT8 MACs, one MAC / PE / cycle.
- ▶ Weight-stationary: *reg_w* loaded once via *set_w*, held for the whole matmul; *x* streams west $\rightarrow$ east, *psum* north $\rightarrow$ south.
- ▶ 20-bit signed accumulator ($2 \cdot \text{data_width} + \log_2 m$, $m = 16$).
- ▶ Output round-half-up on an arithmetic right shift, then clip to $[-128, 127]$ (Eq. 1, *right_shifter.v*).
- ▶ Runtime block-count registers set the contraction / output dims.

Core architecture - fixed-point non-linear engines



- ▶ Softmax - 3-pass scalar FSM, one buffered row at a time; each pass streams the full row once.
- ▶ Pass 2/3 use a base-2 exp approximation ($\log_2 e$ -scaled); pass 3 avoids a divider with a log-domain term ($\ln_module$).
- ▶ GELU - two-segment piecewise-linear: saturate outside a threshold, linear correction ($\ln.v$) inside.

Per-pass equations (exact base-2 restatement of softmax)

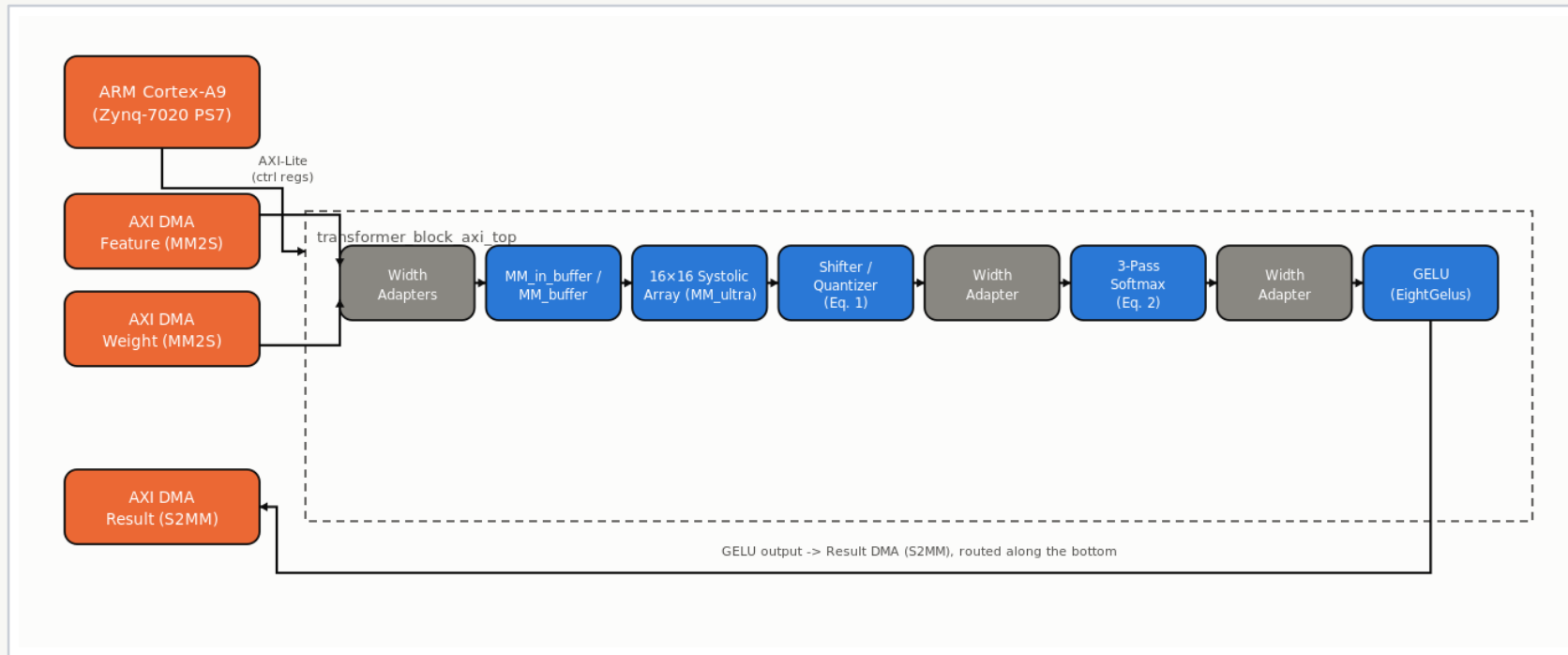
Pass 1 $m = \max_i (x_i)$

Pass 2 $S = \sum_i 2^{(x_i - m) \cdot \log_2 e} = \sum_i e^{x_i - m}$

Pass 3 $y_i = 2^{((x_i - m) - \ln S) \cdot \log_2 e} = \frac{e^{x_i - m}}{S}$

3N cycles / row, strictly serial → the Softmax throughput bottleneck.

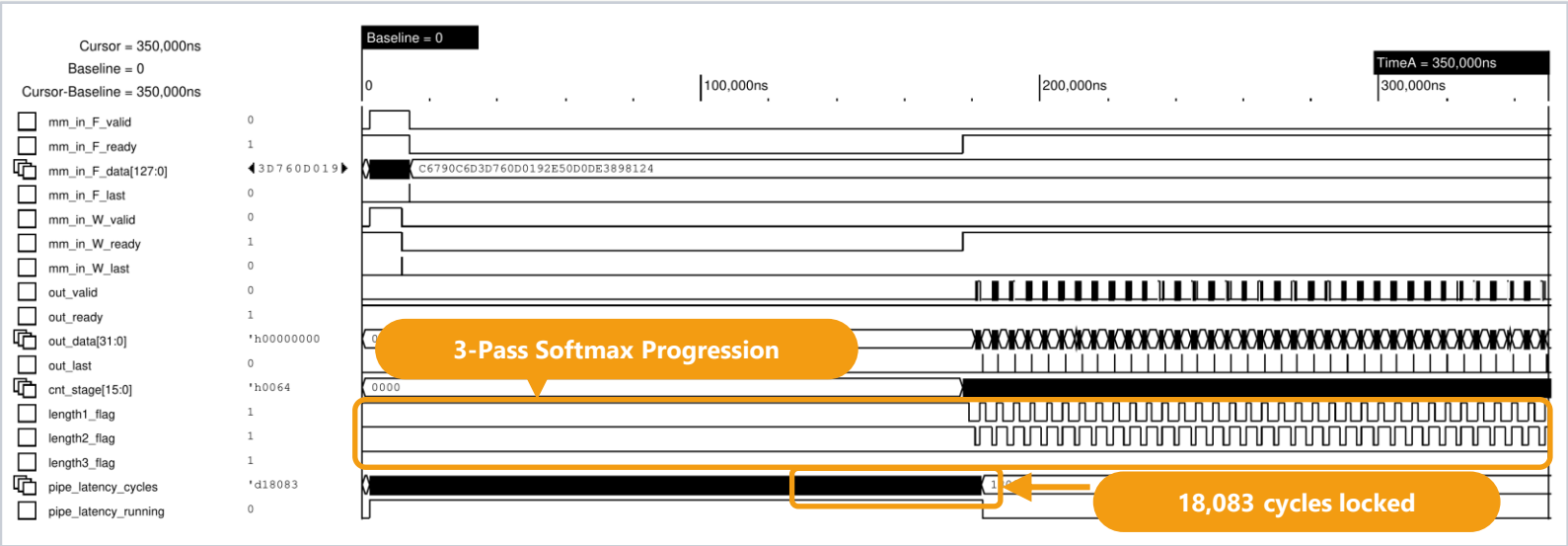
SoC integration on the Zynq-7020



- ▶ AXI4-Lite control (7 config regs + 1 status bit) + dual AXI4-Stream in (feature, weight) / single AXI4-Stream out (result).
- ▶ Three independent AXI DMA engines; PS7 dual-core Cortex-A9 configures and drives the streams; AXI SmartConnect interconnect.
- ▶ Bespoke AXI-Stream width adapters bridge the parallel array ↔ serial Softmax ↔ GELU datapath widths, plus a purpose-built row-boundary signal.

RTL verification - tolerance-bounded, end to end

PROJECT 309



FULL-SCALE RUN · 200 × 96 × 160

0 / 32,000
output elements outside the tolerance bound

18,083
cycle end-to-end latency, read straight off the counter

Real SimVision capture (transformer_block_tb): AXI-Stream ingest, 3-pass Softmax flags, streaming output; pipe_latency_cycles locks at 18,083.

4 SystemVerilog testbenches (gelu, Softmax, MM_Ultra, integration), each with a \$real golden reference.

Technical contribution - the symptom

Post-route, the matmul-dominated accelerator was LUT-bound, not DSP-bound - the exact inverse of what a 256-MAC systolic array should look like.

Resource (post-route, full SoC)	Usage	Reading
DSP48E1 hard multipliers	13 / 220 (5.9%)	almost idle
Slice LUTs	30,276 / 53,200 (56.9%)	near the limit
CARRY4 primitives	4,874	MACs built from fabric

- ▶ A 16×16 INT8 multiply-accumulate array is precisely the workload DSP48E1 blocks exist for - yet synthesis was building the multipliers out of general LUT / CARRY4 logic.
- ▶ First fix attempt: put a (* use_dsp = "yes" *) attribute on the multiply-accumulate always block. Re-synthesis produced byte-identical LUT / DSP / CARRY4 counts - no error, and no effect.

Technical contribution - root cause & fix

Per Xilinx UG901, **use_dsp must attach to the register declaration that holds the multiply result** - an attribute on the always statement is not a recognised attachment point and is dropped during elaboration with no warning.

BEFORE - no-op

```
(* use_dsp = "yes" *)
always @(posedge clk) begin
    psum_out <= psum_in + x_in*reg_w;
end
// attribute on the block: silently ignored
```

AFTER - UG901-correct

```
(* use_dsp = "yes" *) reg signed
    [2*data_width+log2_array_m-1:0] psum_out_r;
always @(posedge clk)
    psum_out_r <= psum_in + x_in*reg_w;
// output reg -> wire, driven by continuous
assign
```

■ keyword ■ (* pragma *) ■ // comment ■ signal / var ■ literal

- ▶ Fix is cycle-for-cycle behaviour-identical: output reg → wire, an internal register carries the attribute, port driven by a continuous assignment.
- ▶ All 256 PEs on DSP would exceed the 220-slice budget, so PE_array.v parameterises the split - NUM_DSP_ROWS = 12 of 16 → 192 PEs on DSP48E1, 4 rows (64 PEs) on LUT/CARRY4.
- ▶ Deliberate, budget-sized partial mapping - not an all-or-nothing switch.

Technical contribution - result at signoff

Metric (post-route, full SoC)	Before	After	Δ
Slice LUTs	30,276 (56.9%)	15,363 (28.9%)	-49.3%
CARRY4 primitives	4,874	1,862	-61.8%
DSP48E1	13 / 220 (5.9%)	205 / 220 (93.2%)	+192
WNS @ 100 MHz	+0.361 ns	+0.293 ns	-19% marg.
Total on-chip power	1.741 W	1.687 W	-3.1%

-49.3%

Slice LUTs

+192

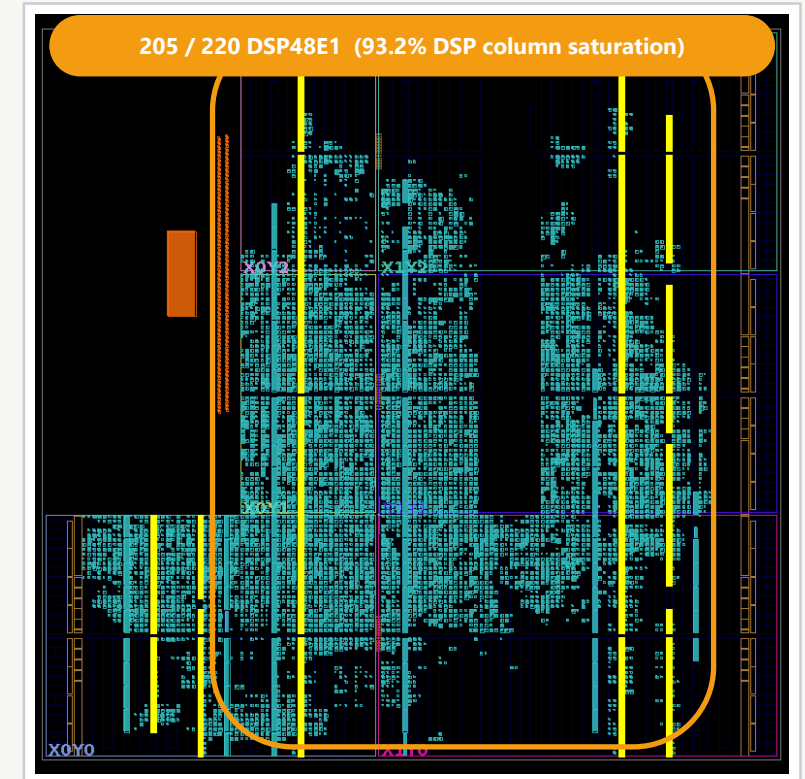
DSP48E1 slices

-3.1%

on-chip power

Lesson a synthesis pragma that raises no error can still do nothing - check utilisation before / after, not just that the build passed.

WNS +0.361 → +0.293 ns: denser DSP column, timing still met - an anticipated trade-off, not a regression.

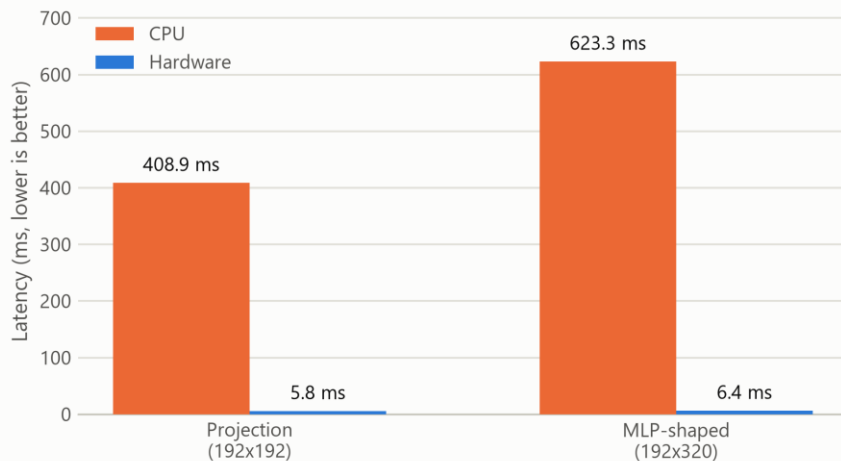


Vivado Device view - post-implementation cell placement across the Zynq-7020 fabric; yellow = DSP / BRAM columns.

Results - hardware kernel vs. ARM Cortex-A9

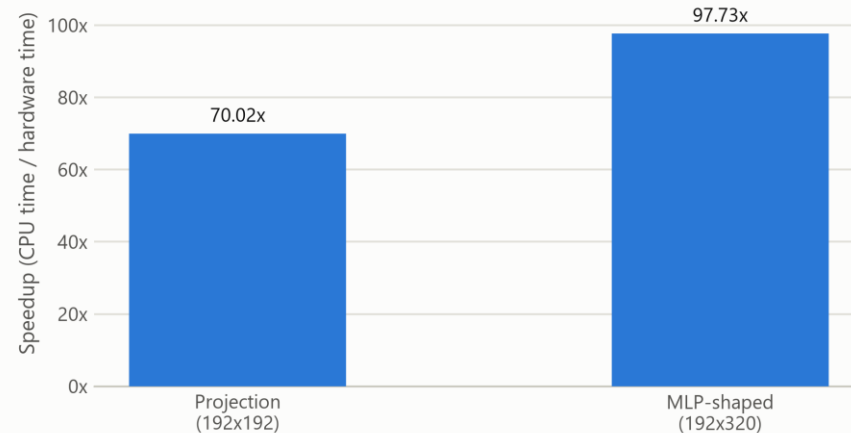
Latency: CPU vs. Hardware Accelerator

ViT-Tiny-representative matmul+softmax+GELU kernel



Hardware Kernel Speedup

Same fused matmul+softmax+GELU operation, hardware vs. CPU



Shape (fused matmul+Softmax+GELU)	HW	CPU	Speedup
192 × 192 (projection-sized)	5.84 ms	408.9 ms	70.02×
192 × 320 (MLP-shaped)	6.38 ms	623.3 ms	97.73×

70.02× - 97.73×

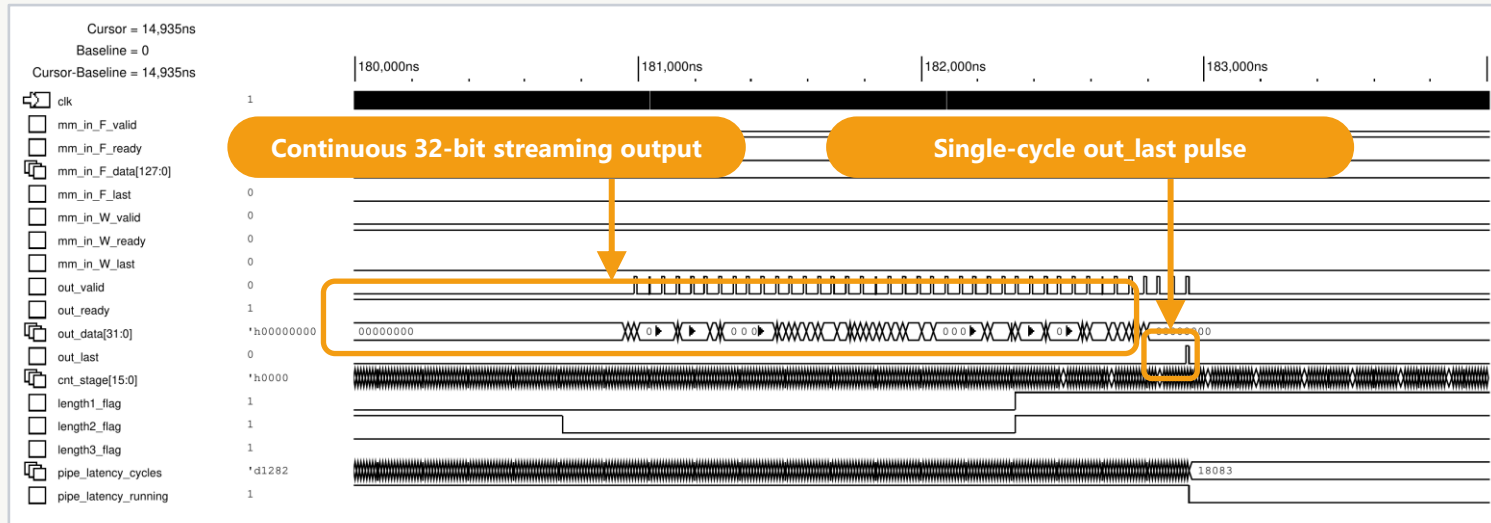
measured kernel speedup vs. ARM Cortex-A9

2.25 GOPS/W

energy efficiency @ 3.795 GOP/s peak (2.47 dyn.)

$$\text{speedup} = \frac{t_{\text{CPU}}}{t_{\text{HW}}} \quad \text{throughput} = 2 \cdot \frac{\text{MNK}}{t_{\text{kernel}}}$$

Results - simulation $\equiv$ silicon



SimVision zoom: end-of-inference handshake - out_valid / out_data[31:0], single-cycle out_last, counter locking at 18,083.

Same shape, real board

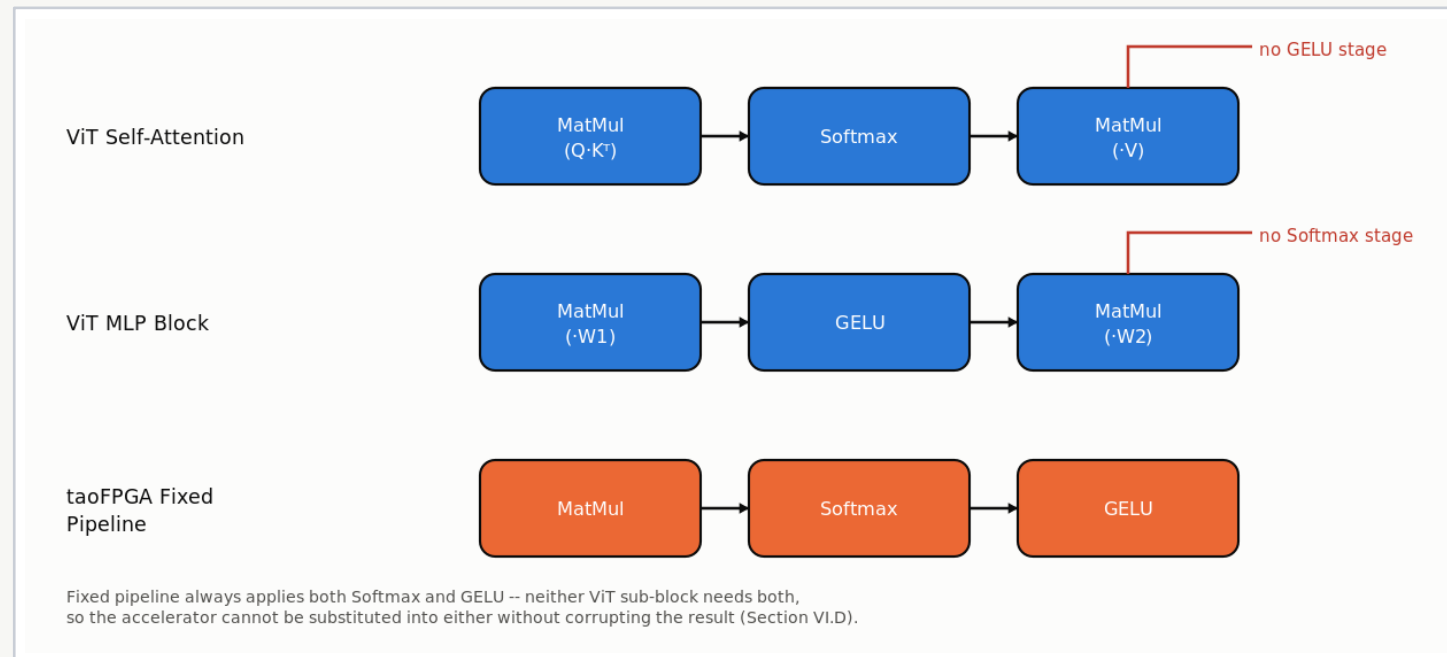
200 × 96 × 160, on the physical PYNQ-Z2, read back over the PYNQ Python driver:

0 / 32,000

output elements outside the tolerance bound

Behavioural simulation and physical silicon agree at the design's originally verified scale - not a new, more forgiving test.

Results - an honest workload analysis



► The fixed matmul→Softmax→GELU pipeline maps to no single ViT operation:

- self-attention = matmul-Softmax-matmul, no GELU
- MLP block = matmul-GELU-matmul, no Softmax

► Substituting it into either would silently apply an extra normalisation / activation and corrupt the result.

► Verified against the RTL, not assumed - so we report a kernel throughput benchmark, never an end-to-end accelerated-ViT claim.

Samoyed · 87.26% confidence



Empirical software-baseline check

From-scratch NumPy ViT-Tiny classifies this sample as *Samoyed* (87.26% confidence) - identical top-1 to the PyTorch reference,

$$\max |\Delta_{logit}| \leq 1.0 \times 10^{-5}$$

$$\Delta_{logit} = |z_{NumPy} - z_{PyTorch}|$$

Comparison with prior FPGA transformer accelerators

Design	Target FPGA (total DSPs)	Precision	DSP util.	Reported efficiency
taoFPGA (this work)	Zynq-7020 (220)	INT8	205/220 (93%)	2.25 GOPS/W; 70-98× vs. on-chip ARM
ME-ViT [3]	Alveo U200 (5,867)	n/s	1,024 (17%)	4.0× power-eff. vs. GPU
FTRANS [4]	VCU118 (6,840)	16-bit fixed + BCM	82-96%	81× / 8.8× vs. CPU / GPU

- ▶ ME-ViT and FTRANS target datacenter-class FPGAs → 25× the DSP budget - and benchmark against GPUs/CPUs.
- ▶ taoFPGA targets a 220-DSP embedded SoC and benchmarks against its own on-chip ARM Cortex-A9.
- ▶ The table situates design choices within one architectural family (fixed-point + DSP-array-dominated, device near DSP saturation) - it is not a device-independent performance ranking.

Planning vs. execution

Phase (pre-project report)	Planned	Actual execution
1 Literature review & theory	Dec 25 · Eliran	Done. Transformer arch, HW quantization, FPGA accel surveyed; 8 refs cited.
2 Architecture & methodology	Jan 26 · Shay	Done. Weight-stationary 16×16 INT8 array + fused fixed-point Softmax/GELU. RTL flow chosen over HLS.
3 Critical component / POC	Feb 26 · Eliran	Done. MM_ultra core built & verified in Cadence Xcelium vs. a SystemVerilog golden model.
4 Full impl. & integration	Apr 26 · Shay	Done, expanded: full pipeline AXI-wrapped from scratch; 60 async-reset blocks fixed / 12 files; 6 stale script params corrected.
5 Performance test & calibration	May 26 · Eliran	Done. PYNQ-Z2 bring-up; full-scale run 0/32,000 errors; 'calibration' became the DSP-inference fix (LUT -49%, power -3%).
6 Results analysis & report	Jun 26 · both	Done. Kernel benchmark vs. on-chip ARM (70-98×). GPU / end-to-end-ViT comparison dropped after workload-mismatch analysis.
Schedule	Final pres. Jun 26	Final report Sep 22 2026 (~3-month slip): early environment/infrastructure debugging + added integration scope.

Challenges & engineering discussion

STANDING BOTTLENECKS

- ▶ 64 KB single-transfer DMA ceiling (16-bit length reg). Blocks the 768-wide ViT-Tiny MLP and all batching.
 - chunking is unsafe: tlast resets the buffer write-address counter every transfer → chunk 2 overwrites chunk 1
- ▶ Serial Softmax: parallel 2-D array (A_size results/cycle) feeding a 1-scalar/cycle, 3N-cycle stage - a structural throughput limit.

REPRESENTATIVE DEBUGGING WINS

- ▶ Async-reset DRC: one root cause chased through 12 files / 3 re-synth rounds → project-wide grep audit.
- ▶ Elaboration-time memory blow-up from a generate-unrolled test array → on-demand packing functions (5.9 GB → 51 MB).
- ▶ A comprehensively stale PYNQ driver, rebuilt from the real register map.
- ▶ Tiny-shape HW mismatch traced to a single-row address-counter edge case - not a design flaw (re-ran the verified shape to confirm).

Improvements & architectural roadmap

1

Scatter-Gather DMA - Replace direct-register DMA (or re-synthesise with a wider transfer-length register). Per-descriptor frame-boundary control removes the 64 KB ceiling → full-width MLP + batching + a real end-to-end path.

2

Decouple the three stages - Matmul / Softmax / GELU as independent streaming nodes with true backpressure; add the missing GELU skid buffer. Addresses the serial-Softmax bottleneck directly.

3

Parallel / pipelined Softmax - Break the 3N-cycle serial structure - the single largest throughput lever, independent of clock or DSP.

4

Correct PYNQ-Z2 board files - Install the vendor board files and re-apply the DDR3 preset (currently PYNQ-Z1-derived, flagged inline).

Logistics - budget, tooling, references

Budget

- PYNQ-Z2 board (Zynq-7020) ~ \$199
- Vivado / Vitis - university / WebPACK license -
- Cadence Xcelium - university license -
- Python / NumPy / PyTorch (timm) free

Total hardware outlay ≈ \$199. No cloud / fabrication cost.

Effort split

- Eliran - literature, matmul core & POC, verification, silicon bring-up & benchmarking
- Shay - architecture selection, full-system integration, AXI wrapper, physical signoff
- Both - results analysis, report & presentations

Key references [1] Vaswani et al., Attention Is All You Need, NeurIPS 2017 · [3] Marino et al., ME-ViT, HiPC 2023 · [4] Li et al., FTRANS, ISLPED 2020 · [6] AMD/Xilinx UG901 (Synthesis) · [7] Dosovitskiy et al., ViT, ICLR 2021

Conclusions - one recommendation per objective

	Delivered	Recommendation
O1 Core + engines	16×16 INT8 array + fused fixed-point Softmax/GELU, formalised as Eq. 1-2.	Decouple the 3 stages into streaming nodes with backpressure.
O2 Verification	0 / 32,000 error at the verified scale, sim $\equiv$ silicon.	Extend the golden-model sweep to small / edge shapes (F_length = 1, 2-row).
O3 SoC bring-up	Runs on real PYNQ-Z2 via the PYNQ driver.	Install vendor PYNQ-Z2 board files; re-apply the DDR3 preset.
O4 Physical signoff	Timing met @ 100 MHz; LUT -49.3%, power -3.1% from one RTL-local fix.	Adopt the register-level use_dsp pattern as the project standard.
O5 Evaluation	70-98× kernel speedup, 2.25 GOPS/W; honest ViT workload analysis.	Scatter-Gather DMA → full-width MLP + batched, end-to-end evaluation.

Governing discipline throughout: verify every layer against its own ground truth, and report every limitation as precisely as the result being claimed.

Thank you

Questions & discussion

Eliran Turgeman · **Shay Rask** · taoFPGA · Project 309

PROJECT 309

Appendix A - key engineering debugging wins

Problem	Root cause	Fix & result
Xcelium session crash at full scale (200×96×160), during native-code generation [A.1]	Testbench x/y/z arrays built with a generate loop - one elaborated hardware object per scalar; memory grew worse than quadratically (59 MB → >5,900 MB at 128 ³).	Replaced with automatic functions pack_x_word / pack_y_word / unpack_z_word (compute each word on demand). ~51 MB / ~4 s, identical 2,894-cycle latency.
40+ post-synthesis DRC warnings: block-RAM control logic driven by asynchronously-reset registers [A.5]	Async reset on BRAM control can corrupt memory outside STA visibility. The pattern recurred one hierarchy level deeper each re-synth - 3 named files became 12.	Project-wide grep for the anti-pattern, then converted 60 always blocks across 12 files to synchronous reset. Bit-for-bit identical simulation after every round.
Softmax_control scale port (4-bit, 7-12) vs. gelu.v scale port (3-bit, max 7) - could only agree at scale = 7 [A.3]	A known, code-commented limitation. gelu.v had no shift branch for scale > 7, so a plain port-width bump would silently mis-shift values 8-15 as if 7 - a correctness bug.	Widened the scale port through all three modules to 4 bits, added the missing shift-direction branch in gelu.v, extended the GELU testbench range 0-6 → 0-11.

Appendix - 3-pass fixed-point Softmax math

Textbook: $\text{Softmax}(x)_i = \frac{\exp(x_i - m)}{S}$ → hardware uses the base-2, log-domain form the exp / Ln units actually compute:

Pass 1 - MAX

$$m = \max_i(x_i)$$

N cycles - running comparator finds the row maximum.

Pass 2 - ACCUMULATE

$$S = \sum_i 2^{(x_i - m) \cdot \log_2 e}$$

N cycles - base-2 exp: split $x \cdot \log_2 e$ into integer + fractional part, linear-interpolate the fractional 2^x term.

Pass 3 - NORMALIZE

$$y_i = 2^{((x_i - m) - \ln S) \cdot \log_2 e}$$

N cycles - second exp unit + Ln_module piecewise log; replaces an explicit hardware divider.

Total 3N cycles / row, sequential. The NumPy golden model uses this identical $\log_2 e$ -scaled form - 'golden' means bit-faithful, not merely close.

Appendix - tolerance-boundary analysis

- ▶ Verification is tolerance-bounded, not exact-match - INT8 requantization and the piecewise exp / log approximations each add sub-LSB error that compounds along the pipeline.
- ▶ **Two INT8 quantization boundaries the error crosses:**
 - ① matmul accumulator → Softmax input: round-half-up + saturate to $[-128, 127]$ (*right_shifter.v*, Eq. 1)
 - ② Softmax output → GELU input: a shared fixed-point scale (`softmax_scale_out = gelu_scale`) so both stages read the boundary identically
- ▶ Unit tests use a tighter, scale-dependent LSB threshold per module; the full-pipeline integration test uses a 6-LSB bound at the shared Softmax-output / GELU scale.
- ▶ 6 LSB absorbs benign quantization compounding while still catching real logic errors - those produce large or structured deviations (e.g. the exact-zero outputs of Appendix A.10), not a uniform $\pm$ few-LSB spread.

6 LSB

integration tolerance at the shared Softmax / GELU scale

0 / 32,000

outside tolerance at full scale ($200 \times 96 \times 160$),
18,083-cycle latency

Same bound is met on real PYNQ-Z2 silicon (Slide 17).